

Input-Output Reasoning

COMPLETE LEARNING GUIDE

What You'll Learn

Input-output (machine input) questions show a starting line of words and/or numbers, then a sequence of 'steps' where the machine rearranges and/or transforms the items according to a FIXED but unstated rule, with each step's output shown — and the question asks either for a specific intermediate step's output or for the original input given a later step. The entire skill is pattern extraction across the shown steps: comparing step 1 to step 2, and step 2 to step 3, to isolate exactly what changed (usually one item's position, or one item's value) at each step, then extending or reversing that same rule. This sub-module covers word-rearrangement machines (usually sorted alphabetically or by word length, one item moved per step), number-rearrangement machines (sorted by value, with an arithmetic operation sometimes also applied), mixed word-number machines, identifying the step-to-step sorting direction, working backward from a given step to find the original input, and how to recognize when a step has 'completed' (no further change).

1. Reading the Step Structure

Before attempting to find the rule, line up every given step vertically, one row per step, so item positions can be compared column by column across consecutive rows — trying to compare steps by reading them left to right in prose, rather than aligned vertically, is the most common reason candidates miss the actual rule.

- Input: 62 train 18 forest 45 apple. Step I: 18 62 train forest 45 apple. Step II: 18 45 62 train forest apple. Aligning these vertically shows: numbers are being sorted in ascending order and moved to the front, one at a time, per step — by Step II, the two smallest numbers (18, 45) have both moved to the front-left.

2. Word-Rearrangement Machines

The most common word-machine rule sorts words alphabetically (or by word length, ascending or descending) and moves the correctly-positioned word to one end per step — identify whether it's alphabetical or length-based by checking the FIRST word moved in step I against all the given words.

- Input: forest apple train jungle. Step I: apple forest train jungle. Comparing to the input, 'apple' moved to the front. Since 'apple' is alphabetically first among all four words, the rule is: bring the alphabetically-earliest remaining word to the front, one word per step.

3. Number-Rearrangement Machines With Arithmetic Twists

Some number machines don't just reorder values but also apply an arithmetic operation (add a constant, multiply by 2) to each number as it's moved — verify this by checking whether the moved number's VALUE changed between steps, not just its position; if unchanged, it's a pure sort, if changed, extract the exact operation from the one example given.

- Input: 12 45 7 30. Step I: 7 12 45 30 (7, the smallest, moved to the front — ascending sort). Step II: 7 14 45 30 (12 was already next in sorted order, so only its value changed: 12!14, a +2 applied when a number is relocated that step). Step III: 7 14 32 45 (30 moved into position 3, becoming 32 with the same +2). The rule: sort ascending one number at a time, adding 2 to a number's value exactly when it's the one being placed that step.

4. Mixed Word-and-Number Machines

When a line contains both words and numbers, the machine typically processes them as two independent streams (e.g., words move to the left end sorted alphabetically while numbers move to the right end sorted numerically, or vice versa) — treat the word-positions and number-positions as two separate mini-machines and solve each stream's rule independently before combining.

- Input: 25 zebra 8 apple. Step I: apple 25 zebra 8. Step II: apple zebra 25 8. Words (apple, zebra) are migrating leftward in alphabetical order while numbers (25, 8) are migrating rightward — two independent streams, sorted in opposite directions, converging from opposite ends.

5. Determining the Sort Direction (Ascending vs Descending)

Compare only the FIRST item moved (step I vs input) against the full original list — if the smallest/alphabetically-first item moved, the sort is ascending; if the largest/alphabetically-last item moved, it's descending. Do not assume ascending by default, since descending-order machines are equally common and deliberately included as a trap for that assumption.

- Input: 30 5 18 42. Step I: 42 30 5 18. The item moved to the front, 42, is the LARGEST number in the list — so the rule is descending order, moving the largest remaining number to the front each step (opposite of the more commonly assumed ascending default).

6. Working Backward: Finding the Original Input

When only a middle or final step is given and the question asks for the original input, run the discovered rule in reverse: identify which item was moved in the LAST shown step transition and undo exactly that one movement (and any associated arithmetic) to reconstruct the previous step, repeating until reaching step 0 (input).

- If the fully-sorted final step reads: apple forest jungle train (alphabetical, one word moved to the front per step, 4 words total means 3 steps to fully sort), and you know 'train' was the LAST word moved (in step III), then in step II 'train' must still have been in its ORIGINAL relative position among the unsorted remainder — reconstructing step by step backward, undoing the most recent move first.

7. Recognizing When Sorting Has Completed

A machine stops applying its rule once every item has reached its final sorted position — recognize completion by checking whether one more application of the discovered rule would produce any change; if applying it again to the last shown step produces an identical row, that step is indeed the final, fully-sorted output and no further steps exist.

Common Pitfalls

- Comparing steps by reading them in prose instead of aligning them vertically column by column, missing the exact single change per step.
- Assuming the sort direction is always ascending without checking whether the first moved item was actually the smallest/alphabetically-first or the largest/last.
- Missing an arithmetic operation applied alongside repositioning (assuming a pure sort when values are also changing between steps).
- In mixed word-number lines, trying to find one unified rule instead of recognizing two independent streams (words and numbers) sorted separately.
- Working backward from a later step without first correctly identifying which single item was moved in the most recent step transition.
- Not verifying the discovered rule against EVERY given step (not just the first one) before applying it to answer the actual question.
- Assuming a machine always moves items to the front, when some machines move the sorted item to the back/end instead.

Key Takeaways

- Always align every given step vertically to compare item positions column by column — never compare steps by reading them as prose.
- Determine sort direction (ascending/descending) by checking whether the FIRST moved item is the smallest/first or largest/last in the original list.
- Check whether item VALUES change (not just positions) between steps — some machines apply arithmetic alongside repositioning.
- In mixed word-and-number lines, solve the word-stream and number-stream rules independently — they usually don't share one combined rule.
- To reconstruct an earlier step from a later one, undo the single most recent move (and any arithmetic) first, then work backward step by step.
- Verify a discovered rule against every given step before extending it forward or backward to answer the question.

Learn More

IndiaBix — Input Output Reasoning

<https://www.indiabix.com/logical-reasoning/input-output/>

GeeksforGeeks — Input Output Reasoning

<https://www.geeksforgeeks.org/input-output-based-reasoning/>